

Introduction to Python

PTP Program



Training Guidelines

Persyaratan dan Ekspektasi untuk Peserta Pelatihan:

1. **Prerequisite:** Peserta diharapkan sudah menginstal [Python](#), [Visual Studio Code](#), dan extension [Jupyter Notebook](#).
 - [Tutorial Instal Python di Windows](#)
 - [Tutorial Instal Python di MacOS](#)
 - [Tutorial Instal Python di Linux](#)
2. **Active Participation:** Peserta harus terlibat aktif dengan materi pelatihan, mencatat poin-poin penting dan penjelasan yang diberikan oleh instruktur.
3. **Hands-on Practice:** Sebagian besar pelatihan akan melibatkan latihan praktis yang dilakukan menggunakan **Google Colab (Jupyter Notebook)**. Anda dapat mengakses Google Colab yang tersedia di slide terakhir.

List Of Contents

- Pengenalan Python
- Python Data Types
- Variables
- Operators and Expressions
- Python Lists
- Python Tuples
- Python Dictionary



Pengenalan Python

Introduction to python

Apa itu Python?

Python adalah bahasa pemrograman *high-level*, *interpreted*, dan *general-purpose programming*. Bahasa ini dikenal karena sintaksisnya yang sederhana dan mudah dibaca, menjadikannya pilihan yang sangat baik baik untuk pemula maupun programmer berpengalaman. Python dikembangkan pada tahun 1980-an oleh Guido van Rossum di National Research Institute for Mathematics and Computer Science di Belanda.

Penggunaan umum Python:

- Pengembangan web (Django, Flask)
- Ilmu data dan pembelajaran mesin (NumPy, Pandas, Scikit-learn)
- Otomatisasi
- Komputasi ilmiah
- Dan banyak lagi!

Introduction to python

Kenapa Memilih Python?

Berikut adalah beberapa alasan mengapa menggunakan Python:

- **Readability:** Sintaksis Python dirancang agar jelas dan ringkas, membuatnya mudah dipelajari dan dipahami.
- **Versatility:** Dari pengembangan web hingga analisis data, Python dapat digunakan untuk berbagai macam tugas.
- **Large community:** Python memiliki komunitas yang sangat besar dan aktif, memberikan dukungan dan sumber daya yang luas.
- **Extensive libraries:** Python Standard Library dan perpustakaan pihak ketiga menawarkan berbagai alat untuk berbagai aplikasi.
- **Python is Free:** Interpreter Python dikembangkan dengan lisensi sumber terbuka yang disetujui OSI, menjadikannya gratis untuk diinstal, digunakan, dan didistribusikan, bahkan untuk tujuan komersial.

Introduction to python

Python Data Types

Sekarang, mari kita bahas tipe data yang tersedia di Python. Python memiliki tipe data berikut yang sudah tersedia secara default, dalam kategori-kategori ini:

- Text:
 - String (str)
- Numeric:
 - Integer
 - Float
 - Complex
- Sequence:
 - List
 - Tuple
 - Range
- Mapping:
 - Dictionary
- Set:
 - Set
 - Frozenset
- Boolean
- Binary:
 - Bytes
 - bytearray
 - Memoryview

Python Data Types

Integers

Di Python 3, sebenarnya tidak ada batasan sejauh mana panjang nilai integer bisa.

Tentu saja, integer tersebut terbatas oleh jumlah memori yang tersedia di sistem Anda, seperti halnya dengan semua hal lainnya, tetapi di luar itu, integer bisa sepanjang yang Anda perlukan.

```
print(123123123123123123123123123123123123123123 + 1)
```

Python mengartikan urutan digit desimal tanpa awalan apapun sebagai angka desimal:

```
print(10)  
print(type(10))
```


Python Data Types

Floating-Point Numbers

Nilai **float** (bilangan pecahan) di Python ditentukan dengan adanya titik desimal. Selain itu, Anda dapat menambahkan karakter **e** atau **E** diikuti oleh bilangan bulat positif atau negatif untuk menyatakan notasi ilmiah (scientific notation):

```
print(4.2)
print(type(4.2))
print(4.)
print(.2)
print(.4e7)
print(4.2e-4)
```

Python Data Types

Strings

String adalah urutan data karakter. Tipe data string di Python disebut **str**.

String literal di Python dapat dibatasi (dikelilingi) menggunakan tanda kutip tunggal (') atau tanda kutip ganda ("). Semua karakter yang ada di antara pembatas pembuka dan penutup yang cocok dianggap sebagai bagian dari string.

```
print("Hacktiv8")
print(type("Hacktiv8"))
print("This string contains a single quote (') character.")
print('This string contains a double quote (") character.')
```

Python Data Types

Boolean

Python 3 menyediakan tipe data **Boolean**. Objek dari tipe Boolean hanya dapat memiliki dua nilai, yaitu **True** atau **False**.

```
print(type(True))  
print(type(False))
```

Ketika Anda membandingkan dua nilai, ekspresi perbandingan akan dievaluasi dan Python akan mengembalikan nilai **Boolean** sebagai jawabannya, yaitu **True** jika kondisi perbandingan terpenuhi, atau **False** jika tidak.

```
print(100 > 200) #False  
print(100 == 200) #False  
print(100 < 200) #True
```

Variables

Variable Assignment

Untuk membuat sebuah variabel, Anda cukup memberikan nilai kepadanya dan kemudian mulai menggunakannya. *Assignment* dilakukan dengan menggunakan tanda sama dengan (=):

```
n = 300  
print(n)
```

Python juga memungkinkan ***chained assignment***, yang memudahkan untuk memberikan nilai yang sama ke beberapa variabel sekaligus:

```
a = b = c = 300  
print(a,b,c)
```

Variables

Variable Names

Secara resmi, nama variabel di Python dapat memiliki panjang berapa pun dan dapat terdiri dari huruf besar dan kecil (A-Z, a-z), angka (0-9), dan karakter garis bawah (_). Satu pembatasan tambahan adalah bahwa meskipun nama variabel dapat mengandung angka, **karakter pertama dari nama variabel tidak boleh berupa angka.**

Contohnya:

```
name = "Hacktiv8"  
Age = 54  
has_laptops = True  
print(name, Age, has_laptops)
```

Variables

Variable Names

Metode yang paling sering digunakan untuk membangun nama variabel yang terdiri dari beberapa kata adalah tiga contoh terakhir ini:

- **Camel Case:** Contoh: `numberOfCollegeGraduates`
- **Pascal Case:** Contoh: `NumberOfCollegeGraduates`
- **Snake Case:** Contoh: `number_of_college_graduates`

Gunakan salah satu dari ketiga metode tersebut yang paling menarik secara visual bagi Anda. Pilih satu dan gunakan secara konsisten.

Operators and Expressions

Apa itu Operators & Expressions?

Di Python, **operator** adalah simbol khusus yang menunjukkan bahwa suatu perhitungan atau operasi harus dilakukan. Nilai-nilai yang diproses oleh operator disebut **operand**. Berikut adalah contoh:

```
a = 10
b = 20
print(a + b)
print(a + b - 5)
```

Operand dapat berupa nilai literal atau variabel yang merujuk pada sebuah objek. Sekumpulan operand dan operator, seperti **a + b - 5**, disebut sebagai **ekspresi**.

Python mendukung banyak operator untuk menggabungkan objek data menjadi ekspresi. Operator ini digunakan untuk melakukan operasi pada operand dan menghasilkan hasil yang dapat digunakan lebih lanjut dalam program.

Operators and Expressions

Arithmetic Operators

Berikut adalah beberapa contoh penggunaan operator aritmatika dalam Python:

```
a = 4
b = 3

print(a + b)
print(a - b)
print(a * b)
print(a / b)
print(a % b)
print(a ** b)
```


Operators and Expressions

Comparison Operators

Berikut adalah beberapa contoh penggunaan operator perbandingan tersebut dalam Python:

```
a = 10
b = 20

print(a == b)
print(a != b)
print(a <= b)
print(a >= b)
```

Operators and Expressions

String Manipulation

```
s = 'foo'
t = 'bar'

# + and * Operators
print(s + t )
print(s * 4)

#Case Conversion
print(s.capitalize())
print(s.lower())
print(s.swapcase())
```

Introduction to Python Lists

List di Python didefinisikan dengan cara menempatkan urutan objek yang dipisahkan koma di dalam tanda kurung siku (`[]`), seperti yang ditunjukkan di bawah ini:

```
a = ['foo', 'bar', 'baz', 'qux']  
print(a)
```

Karakteristik dari **list** di Python adalah sebagai berikut:

- List berurutan (**order**): List di Python mempertahankan urutan elemen sesuai dengan urutan saat list tersebut dibuat. Elemen list dapat diakses berdasarkan **indeks**.
- List dapat berisi objek apa pun (integer, string, dll).
- List dapat bersarang (**nested**) hingga kedalaman tak terbatas.
- List bersifat **mutable**: List dapat diubah setelah dibuat, misalnya dengan menambah, menghapus, atau mengganti elemen.
- List bersifat **dinamis**: Ukuran list dapat berubah secara otomatis sesuai dengan penambahan atau pengurangan elemen.

Python Lists

Modifying List Value

Sebuah nilai tunggal dalam list dapat diganti dengan menggunakan indeks dan penugasan sederhana:

```
a = ['foo', 'bar', 'baz', 'qux', 'quux', 'corge']  
a[2] = 10  
a[-1] = 20  
print(a)
```

Bagaimana jika Anda ingin mengubah beberapa elemen yang berurutan dalam sebuah list sekaligus?

Python memungkinkan ini dengan **penugasan irisan** (slice assignment), yang memiliki sintaksis sebagai berikut:

```
a = ['foo', 'bar', 'baz', 'qux', 'quux', 'corge']  
print(a[1:4])  
a[1:4] = [1.1, 2.2, 3.3, 4.4, 5.5]  
print(a)
```

Introduction to

Python Tuples

Tuple didefinisikan dengan menempatkan elemen-elemen di dalam tanda kurung biasa `()` alih-alih tanda kurung siku `[]`.

Tuple bersifat **immutable**, yang berarti elemen-elemennya tidak dapat diubah setelah tuple dibuat. Berikut adalah contoh singkat yang menunjukkan definisi tuple, pengindeksan, dan pemotongan (slicing):

```
t = ('foo', 'bar', 'baz', 'qux', 'quux', 'corge')
print(t)
print(t[0])
print(t[-1])

# packing and unpacking
(s1, s2, s3, s4) = ('foo', 'bar', 'baz', 'qux')
print(s1)
```

Introduction to

Python Dictionary

Dictionary di Python adalah sebuah tipe data koleksi yang menyimpan pasangan **key-value** (kunci-nilai). Setiap elemen dalam dictionary terdiri dari **key** (kunci) yang unik, dan **value** (nilai) yang terkait dengan kunci tersebut.

Dictionary dan **list** memiliki beberapa **kesamaan** sebagai berikut:

1. **Keduanya bersifat mutable**
2. **Keduanya bersifat dinamis**
3. **Keduanya dapat dibuat bersarang (nested):**
 - Sebuah list dapat berisi list lain.
 - Sebuah dictionary dapat berisi dictionary lain.
 - Sebuah dictionary dapat berisi list, dan sebaliknya.

Dictionary berbeda dari **list** terutama dalam cara elemen-elemen diakses:

- Elemen **list** diakses berdasarkan **posisinya** dalam list, melalui **indeks**.
- Elemen **dictionary** diakses melalui **kunci** (keys).

Python Dictionary

Defining & Accessing Dictionary

Anda dapat mendefinisikan sebuah **dictionary** dengan menempatkan pasangan **key-value** yang dipisahkan koma di dalam tanda kurung kurawal (`{}`). Tanda titik dua (`:`) digunakan untuk memisahkan setiap **key** dari **value** yang terkait.

```
MLB_team = {  
    'Colorado': 'Rockies',  
    'Boston': 'Red Sox',  
    'Minnesota': 'Twins',  
}  
  
print(MLB_team['Minnesota'])  
#Adding an entry to an existing dictionary  
MLB_team['Kansas City'] = 'Royals'  
MLB_team
```

Python Dictionary

Building a Dictionary Incrementally

Anda bisa mulai dengan membuat sebuah **dictionary** kosong, yang didefinisikan dengan kurung kurawal kosong (`{}`).

Kemudian, Anda dapat menambahkan **key** dan **value** baru satu per satu.

```
person = {}
person['fname'] = 'Hack'
person['lname'] = '8'
person['age'] = 51
person['spouse'] = 'Edna'
person['children'] = ['Ralph', 'Betty', 'Joey']
person['pets'] = {'dog': 'Fido', 'cat': 'Sox'}
print(person)
print(person['children'][1])
```




External References

Colab Link

[Visit Here](#)



Hacktiv8id



Hacktiv8id



Hacktiv8



Hacktiv8 Indonesia